🐳 Running Java JDK Inside a Docker Container – Creative Theory Notes

1. Why Run JDK Inside a Docker Container?

Traditionally, running Java applications requires installing the **JDK directly on the host machine**. This can lead to:

- Version conflicts
- Environment mismatch across systems
- Repeated setup on different machines

Docker solves this by allowing developers to **package the Java application along with the JDK runtime inside a container**, ensuring consistency and portability.

---

2. Introduction to JShell (Java REPL)

JShell is a Java **Read–Eval–Print Loop (REPL)** introduced in **Java 8+**.

Key features:

- Allows interactive execution of Java code
- Useful for testing small code snippets
- Requires a full JDK installation

The goal in this segment is to achieve the **same JShell functionality inside a Docker container**, without installing JDK locally.

---

3. Initial Docker Environment Check

Before proceeding:

- No Docker containers are running
- No Docker images are present

This clean state helps demonstrate the complete workflow:

- Searching images
- Pulling images
- Running containers

---

4. Searching for JDK Images on Docker Hub

Docker Hub provides multiple Java-related images.

Common options include:

- **OpenJDK (official image)**
- **Eclipse Temurin (formerly AdoptOpenJDK)**

The speaker emphasizes choosing **official and trusted images**, with OpenJDK being the preferred choice for demonstrations.

---

5. Understanding Image Tags and Versions

Docker images come with **tags** that represent:

- Java version (e.g., 8, 11, 17)
- OS base (e.g., slim, alpine)
- Architecture compatibility

Selecting the correct tag ensures:

- Compatibility with application requirements
- Optimal image size and performance

---

6. Architecture Compatibility (Very Important)

Modern systems use different CPU architectures:

- **ARM64** → Apple Silicon (M1 / M2)
- **AMD64 / x86_64** → Windows, Linux servers, Intel Macs

Docker images must match the **host machine architecture**, otherwise the container may fail to run. Always verify supported architectures on Docker Hub before pulling an image.

---

7. Pulling the OpenJDK Docker Image

After selecting the correct tag:

- The OpenJDK image is pulled from Docker Hub
- Image download progress is displayed
- Image is verified locally using Docker commands

At this stage:

- Image exists locally
- No container is running yet

---

8. Running the JDK Container (First Attempt)

When the container is run normally:

- It starts
- Executes its default command
- Exits immediately

This happens because:

- No interactive session is specified
- The container completes its task and stops

This behavior highlights how containers are **task-oriented and lifecycle-driven**.

---

9. Interactive Mode with `-it` Flag

To keep the container running and accept user input, Docker provides **interactive mode**.

Using the `-it` option:

- `-i` → Interactive input
- `-t` → Allocates a terminal

This keeps the container in the foreground, allowing users to type commands inside it.

---

10. Using JShell Inside the Container

Once inside the running container:

- JShell is launched
- Java commands are executed interactively
- Variables are created
- Output like **"Hello World"** is printed

This confirms that:

- JDK is successfully running inside Docker
- Java code can be compiled and executed without local JDK installation

---

11. Container as a Java Runtime Environment

The running container now behaves like a **complete Java environment**.

It can be used to:

- Run simple Java programs
- Execute compiled applications
- Host complex frameworks like **Spring Boot**

This demonstrates Docker's strength in isolating and standardizing runtime environments.

---

12. Preparing for Real-World Applications

With JDK running inside Docker, developers can:

- Build and deploy Java applications consistently
- Package dependencies with the runtime
- Avoid environment mismatch issues

The next logical step is deploying a **Spring Boot web application** inside a Docker container.

---

13. Conclusion

Running Java JDK inside a Docker container eliminates the need for local JDK installation while providing a consistent and portable runtime. By using official OpenJDK images and interactive Docker modes, developers can seamlessly run Java applications—from simple scripts to enterprise-grade systems—inside containers.

This forms the foundation for **containerized Java application development**.

If you want next, I can:

- Walk through **Dockerizing a Spring Boot app**
- Explain **Dockerfile for Java applications**
- Create **step-by-step lab notes**
- Summarize this as **exam-oriented short answers**
- Compare **JDK on host vs JDK in container** ☕🚀